that test passes, and keep going. For a slightly more extreme version, check out Joel Turnbull's talk from RailsConf 2014.[8]

Common Rails Gotchas

The following patterns in Rails lead to relatively silent test failure. I stumble over them all the time.

- ActiveRecord models don't save when I expect them to. The most common cause of this problem is that the creation of the object fails a validation. Often this causes a test failure down the line because a record that was supposed to be found in the database isn't there—because it didn't save. Using a factory tool helps with this, as does using `save!` and `create!` in test setup to have the failure happen at the point of the code problem and not further down the line.
- In versions of Rails that use `attr_accessible`, Rails silently ignores inaccessible parameters in mass assignment. Rails 5 will still ignore parameters not specified in a strong parameter `permit` command, but it'll at least log them. This can lead to hours of fun as you try to figure out why that attribute, which is clearly part of the method call, is not part of the resulting object.
- In integration tests, it's common to forget to log in when required. This normally means anything you expect to be on the resulting page fails to show up. The `save_and_open_page` command is invaluable here since it's sometimes hard to tease out what happened from the log. But `save_and_open_page` will clearly show you that you're not logged in.

What You've Done

In this chapter you looked at tools you can use to diagnose a failing test. You can print values to the console using the Ruby `p` method or the Awesome Print gem. If you're having trouble isolating when a breaking change was introduced, you can use `git bisect` to isolate which change caused the problem. If you think you have a test failure caused by the interaction of multiple specs, `rspec bisect` or `minitest bisect` is your friend. If you want to walk through your code step by step, the Pry gem offers a number of tools to help you do exactly that.

Troubleshooting is only one way to improve your test experience. You can also make your test setup better in a more general way, as I'll discuss in the next chapter.

8. http://www.confreaks.com/videos/3365-railsconf-debugger-driven-developement-with-pry